

Struktury Danych i Ich Złożoność w Języku Python

Podręcznik inżynierski dla systemów przetwarzania danych

Wybór odpowiedniej struktury danych jest kluczowym elementem optymalizacji wydajnościowej oprogramowania. W języku Python wbudowane struktury danych są wysoce zoptymalizowanymi obiektami napisanymi w języku C. Zrozumienie ich wewnętrznej implementacji pozwala na uniknięcie pułapek wydajnościowych związanych z niewłaściwą złożonością obliczeniową operacji.

1. Listy (List)

Lista w Pythonie jest dynamiczną tablicą wskaźników. Oznacza to, że elementy listy są przechowywane w ciągłym bloku pamięci, a sama lista przechowuje referencje do obiektów, a nie ich bezpośrednie wartości. Zapewnia to szybki dostęp do dowolnego elementu za pomocą indeksu.

Charakterystyka:

Listy są mutowalne (można zmieniać ich zawartość). Kiedy brakuje miejsca na nowe elementy, Python alokuje większy ciągły blok pamięci (zazwyczaj nadmiarowo, aby uniknąć częstych reallokacji) i kopiuje stare wskaźniki.

```
# Przykład operacji na listach
owoce = ["jabłko", "banan", "gruszka"]
owoce.append("pomarańcza") # Złożoność amortyzowana O(1)
owoce.insert(1, "truskawka") # Złożoność O(n) - wymaga przesunięcia elementów
pierwszy = owoce[0]          # Złożoność O(1) - bezpośredni dostęp
```

2. Krotki (Tuple)

Krotka to struktura bliźniaczo podobna do listy, jednak z jedną kluczową różnicą: jest całkowicie niemutowalna (immutable). Po utworzeniu krotki nie można zmienić jej rozmiaru ani podmienić referencji do jej elementów.

Zastosowanie i zalety:

Ponieważ krotki są niezmiennicze, Python optymalizuje je pod kątem zużycia pamięci. Alokowana jest dokładnie taka ilość pamięci, jaka jest potrzebna. Krotki mogą być również używane jako klucze w słownikach, o ile zawierają wyłącznie inne obiekty niemutowalne.

```
# Definicja krotki reprezentującej punkt geograficzny
wspolrzedne = (52.2297, 21.0122)
# próba wykonania: wspolrzedne[0] = 53.0 zakończy się błędem TypeError
```

3. Słowniki (Dictionary)

Słownik to implementacja tablicy mieszającej (Hash Map). Przechowuje pary klucz-wartość, gdzie każdy klucz musi być unikalny i musi być obiektem hashowalnym (posiadającym niezmienną wartość hash w czasie swojego życia).

Mechanizm działania:

Funkcja skrótu (hash function) przekształca klucz w indeks numeryczny, pod którym w pamięci zapisywana jest referencja do wartości. Dzięki temu wyszukiwanie elementu, dodawanie oraz usuwanie odbywa się w stałym czasie średnim.

```
# Słownik jako baza danych użytkownika
user = {
    "id": 1024,
    "username": "jkowalski",
    "email": "jan.kowalski@domain.com"
}
# Wyszukiwanie użytkownika po kluczu
if "username" in user: # Operacja O(1)
    print(user["username"])
```

4. Zbiory (Set)

Zbiór to nieuporządkowana kolekcja unikalnych elementów. Podobnie jak słownik, zbiór bazuje wewnętrznie na tablicy mieszającej, co wyklucza przechowywanie duplikatów oraz obiektów niemutowalnych jako elementów zbioru.

```
# Eliminacja duplikatów z listy za pomocą zbioru
numery = [1, 2, 2, 3, 4, 4, 4, 5]
unikalne_numery = set(numery) # {1, 2, 3, 4, 5}
# Operacje mnogościowe
zbior_a = {1, 2, 3}
zbior_b = {3, 4, 5}
iloczyn = zbior_a.intersection(zbior_b) # {3} - złożoność O(min(len(A), len(B)))
```

5. Zestawienie Złożoności Obliczeniowej

Poniższa tabela przedstawia porównanie teoretycznej złożoności obliczeniowej w notacji dużego O dla podstawowych struktur danych w Pythonie. Reprezentuje ona najczęstsze operacje wykonywane w pętlach programistycznych.

Struktura danych	Dostęp po indeksie / kluczu	Dodanie elementu (na koniec)	Wyszukiwanie elementu (wartości)	Usunięcie elementu
Lista (List)	O(1)	O(1)	O(n)	O(n)
Krotka (Tuple)	O(1)	N/A	O(n)	N/A
Słownik (Dict)	O(1)	O(1)	O(1)	O(1)
Zbiór (Set)	N/A	O(1)	O(1)	O(1)

6. Podsumowanie dla Parserów Fiszek

Analizując ten dokument za pomocą parserów tekstu w celu ekstrakcji fiszek, warto zwrócić uwagę na regularność nagłówków typu H2 i H3 jako głównych pojęć oraz fragmentów oznaczonych pogrubieniem jako bezpośrednich definicji. Kody źródłowe w blokach zawierają konkretne implementacje metod, które idealnie nadają się na pytania typu: "Jaka jest złożoność metody `.append()`?".